

.NET BACKEND DEVELOPMENT

Self-Study Progress Report — Part III

ASP.NET Core MVC | JWT Authentication | CRUD Operations | React Frontend Integration

Prepared by	Dereje Abtew
Position	Application Management Officer (Trainee Banker)
Report Date	May 4, 2026
Study Reference	Rohan Kumawat — ASP.NET Core MVC .NET 10 Full Course (YouTube, 3 hrs)
Continuation of	Part I (April 27, 2026) & Part II (April 30, 2026)

1. Introduction

This report documents the third phase of my self-directed .NET backend development training. Unlike Parts I and II — which focused on building a Minimal API with in-memory and then database-backed data using the GameStore.Api project — this phase introduces a significantly more advanced and comprehensive set of concepts studied through the ASP.NET Core MVC Full Course by Rohan Kumawat (YouTube, 3 hours, March 2026).

In this phase, I successfully understood and implemented: the full MVC (Model-View-Controller) architectural pattern, SQL Server integration with Entity Framework Core, JWT (JSON Web Token) Authentication and Authorization, cookie-based authorization, a complete CRUD application with Login and Registration pages, and the data-passing mechanisms available in ASP.NET Core (ViewBag, ViewData, TempData). I also began work on a User Registration application integrating a React.js frontend with an ASP.NET Core backend, which represents a major milestone in my full-stack development journey.

2. Key Concepts Studied

The following table summarizes the major new concepts introduced in this phase of study:

Concept	Explanation
ASP.NET Core MVC	A web application framework that organises code into three components: Model (data), View (UI), and Controller (logic). It is more structured than Minimal API and is suited for full web applications that render HTML pages.
MVC Controller vs API Controller	An MVC Controller returns Views (HTML pages rendered with .cshtml templates). An API Controller returns data (JSON/XML) for consumption by frontend frameworks like React. Both are used in different contexts within the same solution.
_layout.cshtml	The shared master page template in ASP.NET Core MVC. All views that reference this layout inherit its header, footer, navigation, and CSS, avoiding duplication across every page.
Routing & Views	Routes map incoming URLs to specific Controller actions. Views are .cshtml files that render the HTML shown in the browser. Each Controller action typically returns a corresponding View.
ViewBag / ViewData / TempData	Three mechanisms for passing data from a Controller to a View. ViewBag uses dynamic properties. ViewData uses a dictionary. TempData survives across one redirect and is then cleared, making it ideal for success or error messages after form submissions.
asp-for Binding	An HTML helper tag attribute used to bind a View's form fields to a C# model class property, enabling automatic data mapping when a form is submitted.
JWT Authentication	JSON Web Tokens are a compact, digitally-signed method for securely transmitting authentication information between a client and server. A token is issued at login and must be included in all subsequent requests to access protected resources.
Cookie-Based Authorization	An alternative to sending the JWT token in headers. The token is stored as a browser cookie, which is automatically included in every request — simplifying implementation for web-based MVC applications.
Password Encryption	Rather than storing plain text passwords, a hashing algorithm is applied so that even if the database is compromised, actual passwords cannot be recovered directly.
Input Validation & Null Handling	Server-side validation rules applied to incoming form data to ensure required fields are present and data formats are correct before processing.
DTO (Data Transfer Object)	As studied in Parts I and II, a DTO filters the data exchanged between client and server, hiding sensitive internal fields and shaping the API response appropriately.
Code-First Approach	EF Core generates the database schema automatically from C# model classes, rather than writing SQL manually. Migrations translate model changes into database updates.

3. Project Setup & Environment

- For this phase of study, I set up a new ASP.NET Core MVC project using Visual Studio 2026 (as recommended in the course) and the .NET 10 SDK. The project setup steps were:
- Installed Visual Studio 2026 with the ASP.NET and web development workload selected.
 - Created a new project using the ASP.NET Core MVC template — distinct from the Empty or Web API templates used in earlier phases.
 - Explored the auto-generated folder structure, including the Controllers, Views, Models, and wwwroot folders, as well as Program.cs and appsettings.json.
 - Reviewed launchSettings.json to understand application URL configuration and browser launch behavior.
 - Installed the following EF Core NuGet packages: Microsoft.EntityFrameworkCore.SqlServer and Microsoft.EntityFrameworkCore.Tools.
 - Installed and configured SQL Server as the database engine — a step up from SQLite used in earlier phases, reflecting an enterprise-grade database system.

 **Visual Studio 2026 vs Visual Studio Code**

This phase uses Visual Studio 2026 (the full IDE) rather than Visual Studio Code (the lightweight editor used in Parts I and II). Visual Studio provides richer MVC scaffolding, built-in Razor view design support, and tighter SQL Server tooling, which is beneficial when working with the full MVC pattern.

4. MVC Architecture

4.1 The MVC Pattern

The Model-View-Controller (MVC) design pattern is the foundation of this phase. It separates an application into three interconnected components, each with a distinct responsibility, making the codebase easier to maintain, test, and scale.

BROWSER / CLIENT – sends HTTP Request
▼ ROUTER (maps URL to correct Controller action)
▼ CONTROLLER (handles logic, calls Model/DB, selects View)
▼ MODEL (C# class – represents data & business rules)
▼ EF CORE (O/RM – translates C# objects to SQL queries)
▼ SQL SERVER DATABASE (stores data persistently)
▲ VIEW (.cshtml – renders HTML response back to browser)

When a user makes an HTTP request, the Router determines which Controller action should handle it. The Controller contains the business logic, interacts with the Model (data layer) as needed, and then selects a View to return to the browser. The View is a .cshtml file that combines HTML with C# expressions to render a dynamic page.

4.2 MVC Controller vs API Controller

A key distinction I clarified in this study phase is the difference between an MVC Controller and an API Controller in ASP.NET Core:

MVC Controller	API Controller
Returns HTML Views (.cshtml)	Returns JSON / XML data
Used in web browser applications	Used by frontend apps (React, Angular)
Inherits from Controller	Inherits from ControllerBase
Uses ViewBag, TempData, ViewData	Uses DTOs and HTTP status codes
Renders Razor pages on the server	Client handles rendering

5. Database Setup with SQL Server & EF Core

5.1 Connection String

I configured the SQL Server connection string in appsettings.json. This is the configuration value that tells EF Core where to find the database and how to authenticate with it:

```
"ConnectionStrings": {
  "DefaultConnection":
    "Server=localhost;Database=EmployeeDB;Trusted_Connection=True;"
}
```

5.2 Creating Models and DbContext

I created C# model classes to represent the application's data entities. For the Employee management application built in this course, the key model included properties such as Id, Name, Department, Email, and Designation. An AppDbContext class was created inheriting from DbContext, exposing DbSet properties for each model to enable EF Core queries.

5.3 Code-First Migration

Using the Code-First approach, I ran the following EF Core migration commands to generate and apply the database schema:

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

EF Core analyzed the model classes and automatically created the corresponding database tables in SQL Server, including all column definitions and relationships.

6. Views, Routing & Data Passing

6.1 Layout and Views

I studied the role of `_layout.cshtml` as the master template shared across all pages. Views are individual `.cshtml` files linked to specific Controller actions. The routing convention in ASP.NET Core MVC follows the pattern `/ControllerName/ActionName`, which maps automatically to the corresponding method in the Controller class.

6.2 View vs RedirectToAction

I learned the important distinction between returning a View (renders a page in place) and returning a RedirectToAction (sends the browser to a completely different Controller action). RedirectToAction is commonly used after form submissions to prevent duplicate POST requests if the user refreshes the page — a pattern known as the Post/Redirect/Get (PRG) pattern.

6.3 Data Passing Mechanisms

I studied and understood the three mechanisms for passing data from a Controller to a View:

Concept	Explanation
ViewBag	A dynamic object that allows passing any data by name from Controller to View within the same request. Not strongly typed — no compile-time checking. Example: <code>ViewBag.Message = "Welcome!"</code> ;
ViewData	A dictionary-based mechanism (<code>ViewData["Key"] = value</code>). Similar to ViewBag but accessed as a dictionary. Both ViewBag and ViewData only live for the duration of the current request.
TempData	Survives across exactly one redirect. Ideal for displaying success or error messages after a form submission that redirects to another page. Automatically cleared after being read once.

6.4 Model Binding with asp-for

I applied the `asp-for` tag helper to bind form input fields directly to C# model properties. When a form is submitted, ASP.NET Core automatically maps the submitted values to the corresponding properties of the model class, eliminating the need for manual field-by-field assignment.

7. Login & Registration Pages

I implemented fully functional Login and Registration pages using ASP.NET Core MVC Views, styled using Bootstrap and CSS. The login page was designed with AI assistance as demonstrated in the course, resulting in a clean, professional user interface.

The screenshot below shows the Login page I produced during this study session:

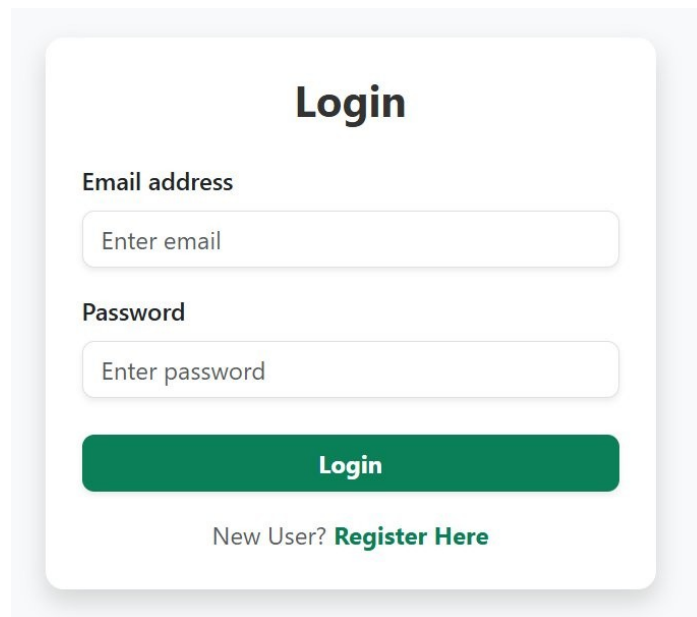


Figure 1 — Login Page: Email address and password fields with a green Login button and Register Here link

The Login functionality performs the following steps in sequence:

- The user enters their email and password in the Login form.
- The Controller receives the submitted data and queries the database via EF Core to find a matching user record.
- The submitted password is hashed and compared against the stored encrypted password hash.
- If valid, a JWT token is generated and stored as a browser cookie for subsequent authenticated requests.
- If invalid, an error message is returned to the View using TempData or ViewBag.

The Registration page collects the user's details, validates all inputs (including null checking), hashes the password before saving, and inserts the new user record into the database.

Password Encryption (Homework Implementation)

Rather than storing plain-text passwords — which would be a critical security vulnerability — I implemented password encryption as an exercise set in the course. A hashing function is applied to the password before it is saved to the database. This means that even if

someone gains access to the database, they cannot reverse the hash to obtain the original password.

8. CRUD Operations — Employee Dashboard

After completing the Login and Registration functionality, I built a full Employee Dashboard that implements all four CRUD operations: Create (Add Employee), Read (list all employees), Update (Edit), and Delete. The dashboard is protected — only authenticated users can access it.

The screenshot below shows the Employee Dashboard I built during this study phase:



Figure 2 — Employee Dashboard: showing Name, Department, Email, Designation columns with Edit and Delete action buttons, and an Add Employee button

The dashboard displays employee records fetched from the SQL Server database via EF Core. Each row includes Edit (blue) and Delete (red) action buttons, and the green '+ Add Employee' button in the top right opens the Create form.

Operation	Route	Controller Action	HTTP Response
Create	POST /Employee/Create	Validates, saves to DB, redirects to list	Redirect 302
Read (List)	GET /Employee/Index	Fetches all employees from DB via EF Core	View 200 OK
Read (Single)	GET /Employee/Edit/{id}	Fetches one employee by ID for editing	View 200 OK
Update	POST /Employee/Edit/{id}	Updates record in DB, redirects to list	Redirect 302
Delete	POST /Employee/Delete/{id}	Removes record from DB, redirects to list	Redirect 302

An important practical insight I gained is that whenever a Model class is modified (for example, adding a new column), a new EF Core migration must be created and applied to keep the database schema synchronized with the updated model.

9. JWT Authentication & Authorization

9.1 What is JWT?

JSON Web Tokens (JWT) are an open standard for securely transmitting authentication and authorization information between a client and a server. A JWT token consists of three Base64-encoded parts separated by dots: the Header (algorithm type), the Payload (user claims, expiry), and the Signature (cryptographic hash to verify authenticity).

9.2 JWT Authentication Flow

The diagram below illustrates the complete JWT authentication flow as implemented in the course application:

Step	Detail	Color Key
1. User submits credentials	POST /login (email + password)	
2. Server validates credentials	Checks DB via EF Core	
3. Server generates JWT Token	Signs with secret key (header.payload.signature)	
4. Token returned to client	Stored in browser cookie or local storage	
5. Client sends Token with requests	Authorization: Bearer <token> header	
6. Server validates Token	Checks signature and expiry on each request	
7. Access granted or denied	Returns protected resource or 401 Unauthorized	

9.3 Cookie-Based Authorization

In addition to the standard Bearer token approach, I studied and implemented cookie-based authorization, which is particularly well-suited to ASP.NET Core MVC web applications. Instead of manually attaching a token to every request header, the JWT is stored in a browser cookie. The browser automatically includes the cookie with every subsequent request to the same domain, making it transparent to the user and simpler to implement in server-rendered applications.

I registered the authentication and authorization middleware in Program.cs and decorated protected Controller actions with the [Authorize] attribute, which ensures that unauthenticated users are automatically redirected to the login page.

JWT vs Cookie Authorization — When to Use Which

JWT Bearer tokens are preferred for API-driven applications where the frontend (React, Angular, mobile app) manages authentication manually. Cookie-based authorization is preferred for traditional server-rendered MVC applications where the browser handles the

token lifecycle automatically. Both were studied and applied in this course.

10. Cross-Controller Redirection

I learned how to redirect a user from one Controller's action to another Controller's action using the RedirectToAction overload that accepts both action name and controller name:

```
return RedirectToAction("Index", "Dashboard");
```

This is essential in the Login flow, where after a successful login, the user must be redirected from the AccountController to the EmployeeController's dashboard — two separate controllers with different responsibilities.

11. User Registration App — React + ASP.NET Core

Building on the knowledge gained in all three phases of this study, I have begun developing a User Registration application that integrates a React.js frontend with an ASP.NET Core backend API. This project represents the convergence of my frontend and backend learning and is currently in progress.

The architectural approach for this full-stack application is:

REACT FRONTEND (User Registration / Login UI – runs in browser)
▼ HTTP Requests (fetch / axios) – JSON payload
ASP.NET CORE API CONTROLLER (handles Register, Login endpoints)
▼ Validates Input Hashes Password Issues JWT Token
EF CORE (O/RM – maps User model to database table)
SQL SERVER / SQLite DATABASE (stores user records securely)

Features implemented so far in the User Registration application include the React-based login form (matching the design shown in Figure 1), the ASP.NET Core API endpoint for user registration, password hashing before database storage, and JWT token issuance on successful login. Frontend-backend integration testing is ongoing.

12. Progress Summary — Part III

The table below summarizes all topics covered in this phase of study:

Topic / Module	Key Achievement	Status
VS 2026 & MVC Project Setup	New MVC project created, folder structure explored	✓ Completed
MVC Architecture	Model-View-Controller pattern understood and applied	✓ Completed
MVC vs API Controller	Differences understood; applied both in practice	✓ Completed
_layout.cshtml	Master template understood and used across views	✓ Completed
SQL Server Setup	SQL Server installed and connected via EF Core	✓ Completed
Connection String Configuration	Correct SQL Server format in appsettings.json	✓ Completed
Models & ApplicationDbContext	Employee model and DbContext created	✓ Completed
Code-First Migration	InitialCreate migration generated and applied	✓ Completed
Views & Routing	URL-to-action routing understood and implemented	✓ Completed
View vs RedirectToAction	PRG pattern applied correctly in form handlers	✓ Completed
ViewBag / ViewData / TempData	All three mechanisms studied and applied	✓ Completed
asp-for Model Binding	Form fields bound to C# model properties	✓ Completed
Login Page (UI + Logic)	Login form built and authentication logic implemented	✓ Completed
Registration Page	User registration with validation and DB save	✓ Completed
Password Encryption	Password hashing implemented before DB storage	✓ Completed
Input Validation & Null Handling	Required field validation applied across forms	✓ Completed
CRUD Dashboard	Full Create, Read, Update, Delete for Employee entity	✓ Completed
JWT Authentication	Token generation, signing, and validation implemented	✓ Completed
Cookie-Based Authorization	[Authorize] attribute and cookie middleware applied	✓ Completed
Cross-Controller Redirection	RedirectToAction with controller parameter used	✓ Completed

Topic / Module	Key Achievement	Status
React + ASP.NET Integration	User Registration app in active development	 In Progress

13. Next Steps

The following areas are planned for the next phase of study:

- Complete the React + ASP.NET Core User Registration application and document it as Part IV.
- Implement role-based authorization (Admin vs regular user) using JWT claims.
- Study CORS (Cross-Origin Resource Sharing) configuration — essential when React runs on a different port from the ASP.NET Core API.
- Explore token refresh logic to handle JWT expiry gracefully.
- Begin studying unit testing for MVC controllers and API endpoints.
- Investigate deployment of a .NET application to a cloud environment such as Azure App Service.

14. Conclusion

This third phase of my .NET self-study represents a major leap in both breadth and depth of knowledge. I have moved from building simple REST APIs to constructing complete, secure, full-stack web applications using ASP.NET Core MVC — a technology that is widely used in enterprise environments, including banking and financial systems.

The skills acquired in this phase are directly applicable to my role as an Application Management Officer. Understanding how enterprise web applications authenticate users, manage data, enforce authorization, and separate responsibilities across layers allows me to engage more meaningfully with the application systems I am responsible for supporting.

I have demonstrated that the knowledge gained is not merely theoretical — I have built and tested working applications, including a Login and Registration system, a full CRUD Employee Dashboard (as shown in the screenshots), and I am actively developing a full-stack React + ASP.NET Core application. I look forward to documenting further progress in Part IV.

Dereje Abteu

Application Management Officer (Trainee Banker)

May 4, 2026